Training Foundational Models From Ground Up

Changhui Hou
University of Southern California

May 2026

Abstract

We present an end-to-end approach for constructing GPT-2 from the ground up using PyTorch, starting with simple character-based bigram models and scaling up through MLP-based approaches, a tinyGPT transformer, and finally a 124M-parameter GPT-2 replica. Each phase of development unpacks core ideas in modern language modeling, from manual backpropagation for bigrams to attention-based decoder-only transformers, achieving competitive perplexities and downstream task accuracy with reduced training overhead. This systematic build-up offers both a framework for understanding GPT architectures and practical insights into efficiently training large transformers under constrained resources.

1 Introduction

Large-scale language models (LLMs) have exploded in popularity and capability in recent years, driven by transformer-based architectures and massive pretraining [Vaswani et al., 2017, Brown et al., 2020]. However, these increasingly complex models demand significant computational resources, making it challenging for individuals with limited hardware to train more advanced models independently.

This project aims to bridge the gap by learning the theoretical underpinnings of LLMs and transformers, and then methodically building a GPT-2-like model from first principles. The journey begins with a character-based autograd engine and simple multilayer perceptrons [Bengio et al., 2003], progresses to convolutional approaches exemplified by WaveNet [van den Oord et al., 2016], and culminates in the transformer framework introduced in *Attention Is All You Need* [Vaswani et al., 2017].

Drawing inspiration from open-source projects such as nanoGPT [Karpathy, 2023], we aim to replicate the GPT-2 architecture [Radford et al., 2019] as precisely as possible. Along the way, we document common pitfalls, optimization tricks, and approaches to make training more efficient. Finally, we aim to surpass GPT-2 performance on challenging benchmarks like HellaSwag [Zellers et al., 2019] while using less computing and fewer tokens. In doing so, we hope to demonstrate the inner workings of GPT-style models and provide a practical roadmap for building them with constrained computing resources.

2 Method and Implementation

2.1 Scalar Autograd

In this section, we outline the step-by-step approach for building and training our minimalist language model with manual backpropagation. As a starting point, we implemented a character-based

bigram model: the model predicts the next character given the immediate previous character (context of one). This toy setup clarifies the fundamentals of a gradient-based learning pipeline, from forward pass to weight updates.

Manual Backpropagation and Autograd Foundation. To gain a deep understanding of how gradients flow through the network, we developed a manual backpropagation algorithm for a simple neural network, and built a graphical function to show a graphical representation of backpropagation. Consider a single neuron with two inputs x_1, x_2, two weights w_1, w_2, and a bias b. The neuron's output, using a hyperbolic tangent activation, is

$$f(x) = \tanh\left(\sum_{i=1}^{2} w_i x_i + b\right).$$

For the bigram language model, we extend this concept to a minimal feed-forward layer receiving character embeddings as inputs. Each parameter—weights and biases—undergoes a manual forward pass followed by an explicit gradient calculation for the backward pass. This approach is inspired by the foundational tutorials in Stanford CS231n.

Character-Based Bigram Model. A bigram language model predicts the next character c_{t+1} given the character c_t. Each character is embedded into a low-dimensional vector representation, and the forward pass outputs logits for the possible next-character tokens. Cross-entropy loss over the predicted distribution and the ground-truth next character measures model performance.

Training Loop. The training loop follows a straightforward pattern:

1. **Forward Pass:** Compute model outputs (logits) given input characters.
2. **Loss Calculation:** Compute cross-entropy loss.
3. **Backward Pass:** Manually compute gradients for each parameter (weights, biases, embeddings).
4. **Parameter Update:** Perform gradient descent updates.

By executing these steps iteratively over a name dataset, the model learns to predict the next character from the current one.

2.2 Bigram `makemore_char`

Building on the names dataset, we first construct a bigram training set by enumerating all (x, y) pairs where x is the current character and y is the next character. Each bigram is counted and stored, yielding a simple dataset for training. This setup allows us to approximate the normalized negative log-likelihood (NLL) of predicting the next character given only a single-character context:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p(y_i \mid x_i).$$

However, bigram models are inherently limited because they only use one character of context. Scaling to trigram or larger contexts with a purely bigram-based approach quickly becomes cumbersome.

2.3 MLP `makemore_char`

To capture more context, we choose a multilayer perceptron model that can look at more than one preceding character simultaneously. Instead of storing transitions for every possible trigram (or higher order n-gram), we feed a fixed-length context window into an MLP. This transforms the problem into a standard feed-forward neural network pipeline, where each input vector encodes multiple characters. By learning distributed representations in a hidden layer, the model can generalize beyond straightforward bigram counting.

Context Window Construction. For a desired context length C, each training sample aggregates the preceding C characters (or special "padding" symbols if fewer than C exist). We then embed each character into a low-dimensional vector and concatenate these embeddings into a single input vector:

$$x = \left[e(c_{t-(C-1)}),\, e(c_{t-(C-2)}),\, \ldots,\, e(c_t)\right],$$

where $e(\cdot)$ denotes a character embedding. The target remains the next character c_{t+1}, cast as a classification problem over the vocabulary.

MLP Architecture. Drawing inspiration from PyTorch's internals [Yang, 2019], we build a single-hidden-layer MLP. Let h denote the hidden layer dimension. The forward pass is:

$$z = \mathrm{concat}\big(e(c_{t-(C-1)}), \ldots, e(c_t)\big), \quad h = \tanh(W_{\mathrm{in}} z + b_{\mathrm{in}}), \quad \hat{y} = W_{\mathrm{out}} h + b_{\mathrm{out}},$$

where $W_{\mathrm{in}} \in \mathbb{R}^{h \times (C \cdot d)}$ maps the concatenated embeddings of dimension $C \times d$ to the hidden layer, and $W_{\mathrm{out}} \in \mathbb{R}^{|\mathcal{V}| \times h}$ outputs logits for each character in the vocabulary $\mathcal{V}$.

Backward Pass and Weight Update. We manually compute the gradients via the chain rule. Specifically, we propagate the loss gradient $\partial \mathcal{L} / \partial \hat{y}$ back through $W_{\mathrm{out}}, b_{\mathrm{out}}$, then through h and into $W_{\mathrm{in}}, b_{\mathrm{in}}$, and finally into the embedding vectors. Parameter updates follow standard gradient descent or variants such as Adam. This approach expands upon the ideas in Bengio et al. [2003] while providing a deeper representation of context compared to the bigram model.

Implementation Details.

- **Dataset Creation:** Convert each name in the dataset into a sequence of character tokens. Extract overlapping windows of length C for input, with target labels as the next character.
- **Embedding Layer:** Use a learnable embedding matrix $\in \mathbb{R}^{|\mathcal{V}| \times d}$, mapping each character ID to a d-dimensional vector.
- **Hidden Layer:** A single fully-connected layer of width h with tanh activation.
- **Output Layer:** A linear layer projecting the hidden representation to logits of size $|\mathcal{V}|$.
- **Loss:** Normalized negative log-likelihood (cross-entropy) for classification over $\mathcal{V}$.

This MLP-based model, dubbed `makemore_char`, can be easily scaled up by increasing the context length C and adjusting the hidden-layer dimension h. This approach avoids enumerating n-grams explicitly and allows for better generalization over unseen character sequences.

2.4 tinyGPT: A Decoder-Only Transformer from the Ground Up

In this subsection, we describe tinyGPT, a decoder-only transformer model built from first principles to generate text from the tinyShakespeare dataset. Our goal is to not only match but surpass the performance of nanoGPT [Karpathy, 2023] on the tinyShakespeare dataset.[1]

Motivation and Baseline. We start with a simple bigram model for next-character prediction. While bigram models offer a straightforward baseline, they fail to capture longer-range dependencies. To incorporate larger context, we leverage a transformer decoder architecture [Vaswani et al., 2017], extending the bigram approach with a multi-head self-attention mechanism.

Attention Mechanisms. Attention allows the model to "focus" on different elements of the context window. We begin with a baseline mean-context approach (equivalent to computing the average of context embeddings), then evolve it into single-head and subsequently multi-head self-attention. Formally, for a given context sequence of length T with embeddings $\{e_1, \ldots, e_T\}$, the self-attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where Q, K, V (queries, keys, values) are linear projections of the input embeddings. Unlike bigrams or n-grams, self-attention captures dependencies between tokens at arbitrary positions, enabling a single pass to model long-range relationships.

Positional Encodings. By design, the attention mechanism is *permutation-invariant* with respect to token ordering. We inject positional information via sinusoidal or learned positional encodings. The resulting token representation is

$$\mathbf{x}_t = e(c_t) + p(t),$$

where $e(\cdot)$ is the token embedding and $p(\cdot)$ is the positional encoding function. This step ensures the model differentiates between different positions in the input sequence.

Layer Normalization. Following standard practice [Vaswani et al., 2017, Wu et al., 2016], each transformer block uses layer normalization before or after attention sublayers; we use the pre-norm formulation. LayerNorm stabilizes training by re-centering and re-scaling intermediate representations.

Decoder-Only Transformer. Our architecture is a stack of L identical decoder blocks without an encoder or cross-attention [Vaswani et al., 2017], akin to GPT-2 [Radford et al., 2019] and ChatGPT [OpenAI]. Each block includes:

1. **Masked Multi-Head Self-Attention:** Ensures each token only attends to itself and preceding tokens.
2. **Feed-Forward Network:** Typically a two-layer MLP with a nonlinearity (e.g., GELU or ReLU) in between.

[1] https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

3. **LayerNorm** and residual connections around each sublayer.

The final linear head projects hidden representations to vocabulary logits. We train via cross-entropy over the next character. We remove the encoder and cross-attention and move LayerNorm before multi-head attention and feed-forward, as shown in Figure 1.

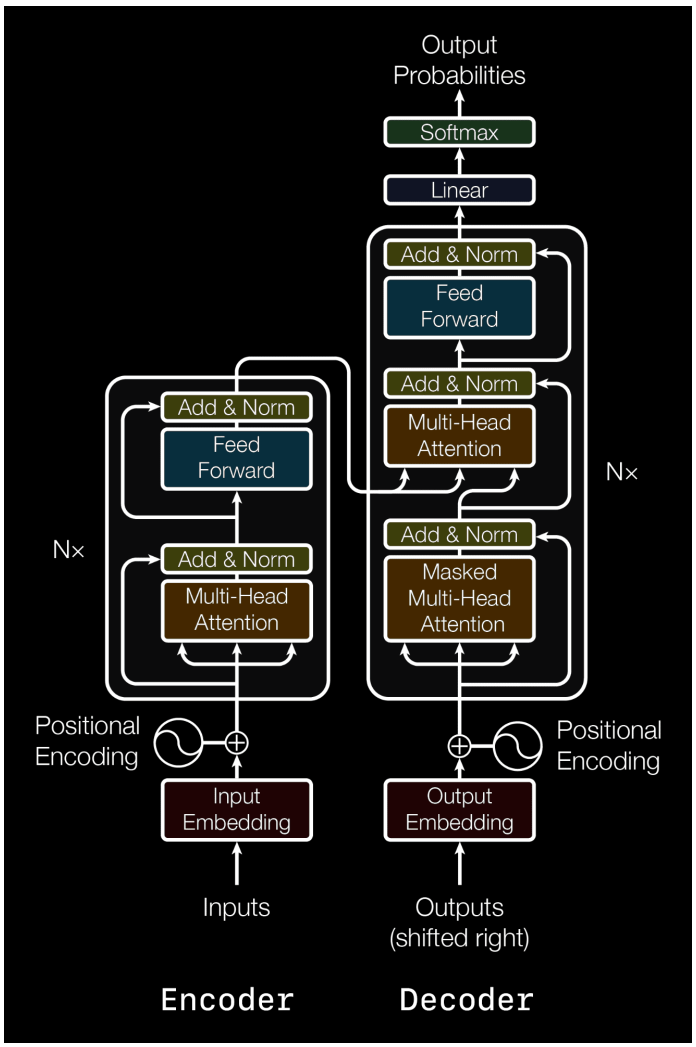

Figure 1: An overview of the Transformer architecture [Vaswani et al., 2017].

2.5 myGPT2: Recreating GPT-2 (124M) in PyTorch

Building upon tinyGPT, we now replicate the 124M-parameter version of GPT-2 as closely as possible in PyTorch. Despite the original GPT-2 codebase [OpenAI, 2019] being vague on specific details, we leverage additional information from GPT-3 [Brown et al., 2020], given the structural similarities between GPT-2 and GPT-3.

Architecture. Structurally, myGPT2 mirrors tinyGPT with multiple transformer decoder blocks. Key hyperparameters include:

$$n_{\text{layer}} = 12, \quad n_{\text{head}} = 12, \quad n_{\text{embd}} = 768, \quad |\mathcal{V}| \approx 50257, \quad \text{block size} = 1024.$$

We load tokens via `tiktoken` to stay consistent with OpenAI's GPT-2. The weight initialization also follows GPT-2's defaults [OpenAI, 2019], ensuring our version remains faithful to the reference model. Each transformer block uses masked multi-head self-attention, a feed-forward MLP, and layer normalization.

Optimizer and Training. We adopt AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, lr = 3e−4, $\epsilon = 1e{-}8$, plus gradient clipping at norm 1.0. A cosine learning rate schedule with warmup further stabilizes training. This approach is inspired by best practices from GPT-3 [Brown et al., 2020] and the NVIDIA Ampere architecture guidelines [NVIDIA, 2020].

Performance Optimizations. The following optimizations yield significant speedups:

- **Automatic Mixed Precision (AMP)** [PyTorch]: Using `torch.autocast()` with TF32 hardware support [NVIDIA, 2020] reduces compute time from ∼310 ms to ∼280 ms per iteration (∼10% speedup).
- `torch.compile()`: PyTorch's compile feature cuts iteration time from ∼280 ms to ∼130 ms (∼50% speedup).
- **FlashAttention-2** [Dao et al., 2022]: Further reduces iteration time to ∼100 ms, a ∼25% improvement. The memory footprint is lowered by avoiding explicit attention matrices in high-bandwidth memory.
- **Vocabulary Size Adjustment:** Changing the vocabulary size from 50257 to a slightly higher power-of-two-aligned 50304 yields a small speedup (∼6%), possibly due to more efficient GPU memory access.
- **Fused AdamW:** Replacing the standard AdamW with a fused kernel version gives an additional ∼4% speed boost.

Distributed Training. Using Distributed Data Parallel (DDP) across 8 GPUs scales throughput from ∼180,000 tokens/sec on a single GPU to ∼1.44 million tokens/sec. This linear scalability highlights the importance of parallelization for large-batch GPT-2 training (∼0.5M tokens per batch).

Discussion. Our myGPT2 model reaffirms that GPT-2 and GPT-3 share a similar underlying design. While the official GPT-2 descriptions are brief, the GPT-3 paper and code provide enough clarity to replicate GPT-2's architecture in PyTorch. The bulk of our effort involves optimization, from specialized hardware features (TF32 cores, fused operations) to advanced self-attention kernels (FlashAttention-2). These practical improvements collectively produce a faster, stable training environment that closely matches OpenAI's GPT-2 performance.

3 Experiments

3.1 tinyGPT

We train on roughly 1M tokens from tinyShakespeare (about 300k tokens in tiktoken units), targeting a model with ∼10M parameters. This training run takes about 10 minutes on a single RTX 4080

GPU, matching the performance of nanoGPT (which in comparison takes about 15 minutes on an A100 GPU with a similar hyperparameter setup). However, tinyGPT is relatively more overfitted.

$$n_{\text{layer}} = 6, \quad n_{\text{head}} = 6, \quad n_{\text{embd}} = 384,$$

$$\text{block size} = 256, \quad \text{batch size} = 64, \quad \text{learning rate} = 0.0003.$$

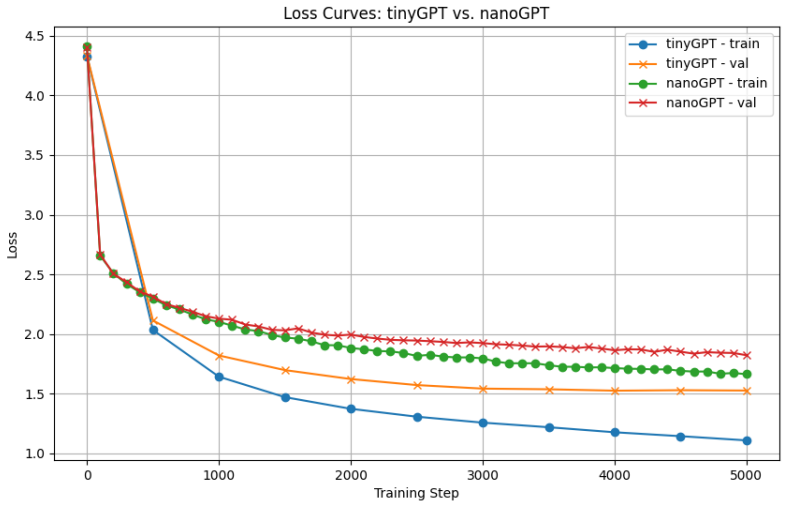

Figure 2: Loss curves on the same dataset and similar size models.

Results and Discussion. Empirically, tinyGPT achieves comparable (and occasionally better) validation perplexities than nanoGPT, underscoring that a carefully implemented decoder-only transformer can replicate the performance of more established GPT-2-like codebases. Our stepwise progression from bigram to multi-head self-attention also highlights how attention-based architectures efficiently learn long-range dependencies and surpass n-gram models.

3.2 myGPT2

The dataset is FineWeb, around 10B tokens. With a set of 8 A100s, myGPT2 trained for one epoch, 19k steps, each step taking around 414 ms, hence around 2 hours of compute on 8 A100s. Key hyperparameters include:

$$n_{\text{layer}} = 12, \quad n_{\text{head}} = 12, \quad n_{\text{embd}} = 768,$$
$$|\mathcal{V}| \approx 50257, \quad \text{block size} = 1024,$$
$$\text{total batch size} = 524288, \quad \text{micro batch size} = 64,$$
$$\text{max lr} = 0.0006, \quad \text{warmup steps} = 715, \quad \text{max steps} = 19073.$$

Besides the optimization experiments shown above, we also evaluate myGPT2 with HellaSwag multiple-choice and completion styles against GPT-2/3. Both myGPT2 and nanoGPT match the performance of GPT-2 with 10 times the data efficiency (Table 1).

Model	Parameters	Training Tokens	Eleuther Harness Acc
GPT-2	124M	100B	0.29
GPT-2 XL	1558M	Unknown	0.40
nanoGPT	124M	10B	0.29
nanoGPT	124M	40B	0.33
myGPT2	124M	10B	0.28

Table 1: Comparison of GPT-like models with Eleuther harness accuracy.

4 Conclusion

In this report, we progressively constructed generative language models of increasing complexity and scale, culminating in a GPT-2 (124M) replication within PyTorch. Our journey—from bigram token predictors and MLP-based `makemore_char` models to tinyGPT and finally myGPT2—highlights the architectural underpinnings and training principles that define state-of-the-art text generation. Along the way, we validated optimization strategies on modern GPUs, demonstrating that techniques such as TF32 mixed precision, `torch.compile()`, and FlashAttention can yield substantial speedups. The final myGPT2 implementation achieves GPT-2-level performance on a fraction of the data tokens, reaffirming that GPT-2 and GPT-3 share a fundamentally similar framework. Moving forward, this foundation can be extended to larger model sizes and more sophisticated optimization pipelines, offering a practical roadmap.

References

Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

Tom Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, et al. Flashattention-2: Further optimizing attention for sequence models. *arXiv preprint arXiv:2307.08691*, 2022.

Andrej Karpathy. nanogpt: Minimalist re-implementation of gpt language model training. https://github.com/karpathy/nanoGPT, 2023.

NVIDIA. Nvidia ampere gpu architecture whitepaper. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf, 2020.

OpenAI. Chatgpt. https://openai.com/index/chatgpt/.

OpenAI. Gpt-2 model code. https://github.com/openai/gpt-2/blob/master/src/model.py, 2019.

PyTorch. Automatic mixed precision examples. https://pytorch.org/docs/stable/notes/amp_examples.html.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. https://cdn.openai.com/

better-language-models/language_models_are_unsupervised_multitask_learners.pdf, 2019.

Stanford CS231n. Convolutional neural networks for visual recognition: Online notes. https://cs231n.github.io/convolutional-networks/.

Aaron van den Oord et al. Wavenet: A generative model for raw audio. In *SSW*, volume 125, page 2, 2016.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. https://arxiv.org/pdf/1607.06450, 2016.

Edward Z. Yang. Pytorch internals. http://blog.ezyang.com/2019/05/pytorch-internals/, 2019.

Rowan Zellers, Yonatan Bisk, Ari Holtzman, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *ACL*, 2019.